ASR1606QuecOpen FOTA
功能参考手册

1 引言

移远通信 cat1 模块支持 QuecOpen 方案；QuecOpen 是开源的基于 Thread-x RTOS 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。本文档以移远通信 1606 平台项目为例介绍 FOTA 功能

本文档主要适用以下模块：

- EC600M-CN
- EC800M-CN
- EC200M-CN

目前 ASR 项目中升级主要有三种：DFOTA(非小系统差分)、FullFOTA(全量升级)、MiniFOTA（小系统升级）。

其中 DFOTA 升级方案制作差分包时又分为两种：FBF 差分以及 adiff 差分，他们两者的区别主要为压缩后差分还是压缩前差分，压缩后差分会比压缩前差分得到的包要大，所以 FBF 差分包占用空间更大；但是它也有其优势，就是 FBF 默认是每 64K 分包差分，那么备份分区 fota_pkg 也就只需要这么大，不像 adiff 差分，是对整个内存小分区进行差分，那么备份时需要的空间就必须要大于包中任意一小分区的大小（1M+），而这个分区平时不使用，也造成了空间的浪费。

MiniFOTA 表述上是小系统差分升级，但实际上做差分的只是小系统代码部分，当小系统差分升级完成后，大系统升级实际上是全量升级（不论是 cp 还是 app 或其他镜像），小系统升级大系统时，下载的大系统全量包会保存在 ram 中（由于处于小系统中，ram 足够大，即使下载远快于升级，也能存下），边下边升，这也就解释了后面制作差分包时升级其他分区为什么直接跟-a+分区名+目标镜像，而不带 old 镜像，且此升级方式没有备份分区 FOTA_pkg。

三种方式对比如下：

功能	DFOTA	minsys	FULL FOTA
差分包存储位置&数量	CUSTOMER_FS (要求空间剩余>2M)	客户无需关注，**2个包** 不占用FS空间	无差分包
全量包存储位置&数量	无	客户无需关注	CUSTOMER_FS (要求空间剩余=APP ROM)
差分包/全量包下载方式	1、客户自行存储到文件系统 2、移远提供一键下载升级【需要开启移远内置的HTTP/FTP功能】	移远提供一键下载升级【HTTP/FTP】	1、客户自行存储到文件系统 2、移远提供一键下载升级【需要开启移远内置的HTTP/FTP功能】
差分包/全量包存储位置	FS（要求客户在下载时>2M）	不占用FS空间	全包大小决定占用空间
是否支持LCD升级进度显示	支持	不支持	不支持
是否支持掉电续升	支持	支持	支持
APP	支持	支持	支持
KERNEL	支持	支持	不支持
是否需要备份	是，fbf方式需要64K，adiff方式以最大分区大小计算，至少1.5M+，使用fota_pkg分区	无	无
重启次数	2次	4次	2次
	要求FS空间有足够空间	重启次数多	速度快

2 FOTA 包的制作流程介绍

升级包制作工具的目录如下图所示：（位于 SDK 中的 ql-bootloader\updater\tool）



名称	修改日期	类型	大小
a	2021/3/2 11:57	文件夹	
b	2021/3/2 11:55	文件夹	
adiff.exe	2021/3/2 10:30	应用程序	149 KB
config	2021/3/2 11:56	.file	1 KB
fbf_dfota.bin	2021/3/2 11:57	BIN 文件	104 KB
fbfmake.bat	2021/3/2 10:30	Windows 批处理...	1 KB
FBFMake_CF_V1.3.exe	2021/3/2 10:30	应用程序	6,379 KB
FBFMake_CF_V1.5.exe	2021/3/2 10:30	应用程序	6,401 KB
FBFMake_CF_V1.6-150.exe	2021/3/2 10:30	应用程序	6,316 KB
fbfmake_fast.bat	2021/3/2 10:30	Windows 批处理...	1 KB
fbfmake_full.bat	2021/3/2 10:30	Windows 批处理...	1 KB
README.docx	2021/3/2 10:30	Microsoft Word ...	119 KB

图：升级包制作工具目录

升级包制作工具的主要目录文件介绍如下，其他目录无需关注：

- 文件夹 *a*（after 的简称）：用于存储新版本固件包中的 *system.img* 文件；
- 文件夹 *b*（before 的简称）：用于存储旧版本固件包中的 *system.img* 文件；
- *config*：用于存储单个分区或者多个分区配置文件；
- *fbfmake_fast.bat*：用于制作旧基线差分升级包；
- *fbfmake_full.bat*：用于制作整包升级包。

通过如下方法完成升级包制作后，将升级包上传至服务器上，在模块上启动 FOTA 功能，即可进行升级包的下载和升级。

2.1 FullFOTA 包制作流程

在 SDK 中首先找到 ql-sdk\ql-bootloader\updater\tool 目录，此目录下是制作所有类型 fota 包均需使用的脚本文件。

步骤一：按需修改此目录下的 config 文件，以下以 customer_app 分区为例：

```
[Image_List]
Number_of_Images = 1 需要制作的分区个数
1_Image_Enable = 1 使能此分区是否参与升级包的制作
1_Image_Image_ID = 0x30
1_Image_Path = customer_app.bin  分区名
1_Image_Flash_Entry_Address = 0x005D7000 分区偏移地址
1_Image_ID_Name = 1
```

图：修改配置文件-制作全量包

步骤二：将应用侧编译生成的 app.bin 重命名为 config 中配置的文件名，此处即 customer_app.bin，并将其拷贝到 tool 目录下的 a 文件夹中。

步骤三：左键双击执行 tool 目录下的 fbfmake_full.bat 脚本文件，执行成功后在升级包制作工具根目录下生成全量升级包 fbf.bin。如下图所示，该升级包比原始镜像增大 8 KB，增大的 8 KB 数据中包含了校验信息和分区定位信息。

R06 › quectel_build › sdk_release › ql-sdk › ql-bootloader › updater › tool

名称	修改日期	类型	大小
a	2022/6/27 19:16	文件夹	
b	2022/6/10 9:39	文件夹	
adiff.exe	2022/6/10 9:39	应用程序	176 KB
adiff_2.3.exe	2022/6/10 9:39	应用程序	120 KB
config	2022/6/27 19:17	文件	1 KB
fbf.bin	2022/6/27 19:17	BIN 文件	35 KB
fbfmake.bat	2022/6/10 9:39	Windows 批处理...	1 KB
FBFMake_CF_V1.2.exe	2022/6/10 9:39	应用程序	6,379 KB
FBFMake_CF_V1.4.exe	2022/6/10 9:39	应用程序	6,399 KB
FBFMake_CF_V1.5.exe	2022/6/10 9:39	应用程序	6,401 KB
FBFMake_CF_V1.6-150.exe	2022/6/10 9:39	应用程序	6,316 KB
fbfmake_fast.bat	2022/6/10 9:39	Windows 批处理...	1 KB
fbfmake_full.bat	2022/6/10 9:39	Windows 批处理...	1 KB

图：全量包

至此，fota 的全量包也就制作完成，不过也是由于全量包是将整个镜像加上校验和分区定位信息组成，所以导致升级包会比原包跟大，而如果想用此来升级 cp、dsp 等较大的镜像，则需要的 ram 必须足够，否则无法保存下载的升级包，**也就不推荐采用全量升级来升级 cp 等较大镜像。**

2.2 FBF 方式差分包制作流程

同样也是找到 SDK 中的 ql-sdk\ql-bootloader\updater\tool 目录，然后执行以下操作：

步骤一：修改 tool 目录下的 config 文件，此流程同全量包中步骤一是一样的，不过正是由于差分包相对较小，所以 config 中配置的镜像可以更多，以 system 和 customer_app 分区进行差分升级为例，需配置信息如下：

```
[Image_List]
Number_of_Images = 2
1_Image_Enable = 1
1_Image_Image_ID = 0x30
1_Image_Path = system.img
1_Image_Flash_Entry_Address = 0x00016000
1_Image_ID_Name = 1
2_Image_Enable = 1
2_Image_Image_ID = 0x31
2_Image_Path = customer_app.bin
2_Image_Flash_Entry_Address = 0x005D7000
2_Image_ID_Name = 2
```

图：修改配置文件-制作差分包

步骤二：将新版本固件包中的 system.img 和 customer_app.bin 文件拷贝至 a 目录下，将旧版本固件包中的 system.img 和 customer_app.bin 文件拷贝至 b 目录下。

步骤三：左键双击执行 fbfmake_fast.bat 脚本文件。执行脚本文件后开始制作差分升级包，脚本执行完 成后会在升级包制作工具根目录下生成差分升级包。

命令详解：
fbfmake_fast.bat 主要执行的命令为：
FBFMake_CF_V1.6-150.exe -f config -d 0x10000 -a a -b b -o fbf_dfota.bin -q
-f：制作差分包需要的配置文件，此处即 config 文件。
-d：代表差分升级使用的切割块大小，一般为 0x10000, 64K。
-a：（after）设置升级后的 image 文件路径，比如 a 文件夹下会放入升级后版本的 system.img。
-b：（before）设置升级前的 image 文件路径，比如 b 文件夹下会放入升级前版本的 system.img。
-o：代表输出的差分文件，为 fbf_dfota.bin
-q: 可选参数，如果选择该参数，差分将选择 fast 模式，制作差分包的时间将大大缩小，但是该模式可能会导致差分包比非 fast 模式要大，在大部分情况下 fast 模式和非 fast 模式输出差分包大小是基本相同。

2.3 adiff 差分制作流程

步骤一：将新旧固件包中的 system.img 拷贝至升级包制作工具根目录下并分别命名为 system_new.img 和 system_old.img；并将新旧固件包中的 customer_app.bin 文件拷贝至升级包制作工具根目录下并分别命名为 app_new.bin 和 app_old.bin（镜像名不是必须同上述保持一致，但

需要与命令中名称一致)。

步骤二：打开命令行并输入执行如下命令开始制作差分包：

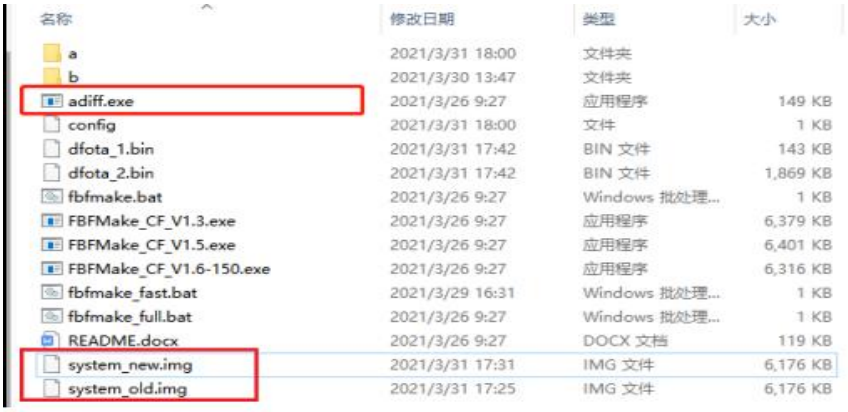
```
adiff.exe -l fp system_old.img system_new.img fbf_dfota.bin
-a1 customer_app app_old.bin app_new.bin
```

执行完成后会在 tool 目录下生成名为 fbf_dfota.bin 的文件，该文件即为制作完成的差分升级包。

2.4 minifota 制作流程

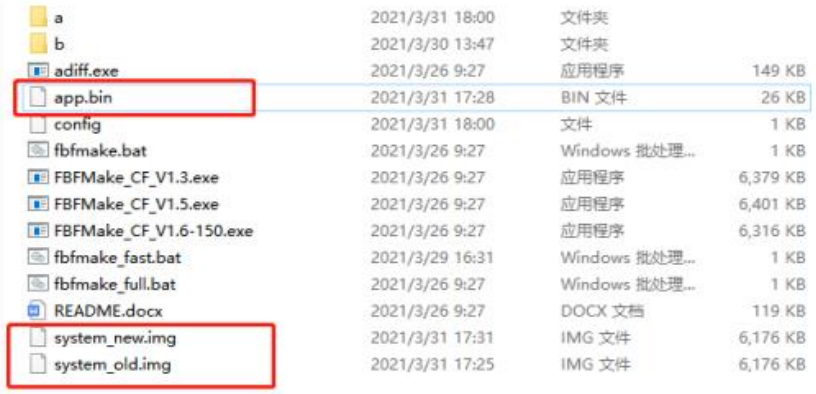
步骤一：拷贝相关文件至升级包制作工具目录。

1. 若仅对 system 分区进行差分升级，将新旧固件包中的 system.img 文件拷贝至升级包制 作工具根目录（需与 adiff.exe 位于同级目录），并分别命名为 system_new.img 和 system_old.img。



图：放置 system.img 文件

2. 如需同时对 system 和其他分区进行差分升级，还需额外将新固件包中的其他分区镜像文 件拷贝至升级包制作工具根目录（需与 adiff.exe 位于同级目录）。以差分升级 system 分 区和 customer_app 分区为例，还需将 customer_app.bin 文件拷贝至升级包制作工具根 目录并重命名该文件为 app.bin。



图：放置 system.img 和 app.bin 文件

步骤二：在 tool 目录下打开命令行，执行如下命令：

```
adiff.exe system_old.img system_new.img
old-new.bin
<Options>
[-a1 <app1_partition_name>
```

前三个参数是必选项，-a1 是可选项，-a2 是可选项。
system_old.img 参数是板子中已经烧写的 system 文件，如 1601 升级包 system.img 重命名为 system_old.img;

system_new.img 参数是要升级到的 system 文件，如 1601 升级包 system.img 重命名为 system_new.img;

old-new.bin 参数是生成的升级文件名称；

-a1 代表当系统中存在和 system 同级的客户分区 app1，如下图所示，分区 app1_partition_name 名称假设为 app1，当要升级这个分区时可以使用这个参数，并将要升级的 app1_file_name 文件放在 **adiff.exe** 同一路径下， app1_file_name 文件在工具制作时会被放入 **old-new.bin_2** 中，在第二阶段升级；
（app1 分区名称客户可以在 aboot 工具下修改，这种分区主要是给客户增加一些特殊的分区，然后也能够升级，使整个最小系统升级更灵活）
-a1 也可以指定为 updater 分区，这样在第二阶段升级就可以升级 updater，命令如下：
adiff.exe system_old.img system_new.img old-new.bin –a1 updater updater.bin

app1_partition_name 代表分区的分名称；
app1_file_name 代表要升级的分区文件；

-a2 同上 **a1**，最小系统升级最多支持同时升级两个 app 分区；（注意 adiff 只能最多添加两个 app 分区用于升级）

小系统升级需要注意的是：调用 api 之后，会先重启进入小系统，然后在小系统中下载新的小系统代码，当下载结束后，继续重启，再升级小系统代码到目标版本的小系统，升级结束后重启，再进入新的小系统中，此时会边下载大系统代码边直接升级大系统代码，升级完成后再次重启后，升级也就结束了。所以 minifota 总共会重启 4 次才会升级成功。

2.5 no updater 分区方案

当前 4+4/4+2 项目中，小系统方案支持 no updater 分区方案。

此方案是将 updater.bin 压缩后放置于差分包的第一个包中，下载后再运行 updater 的功能，而不再在 flash 上划定一块特定的仅用于 fota 时才使用的 updater 分区。

采用此方案时，使用的 adiff 工具必须是 verion: 5.6，Dec 2 2022 / 10:32:17 或更高的版本，使用的命令为：

```
adiff.exe system_old.img system_new.img old_new.bin -u updater.bin
```

或者同样需要升级 app 时，指令为：

```
adiff.exe system_old.img system_new.img old_new.bin -a1 customer customer_app.bin -u updater.bin
```

同之前有 updater 分区的 minifota 方案相比，命令中需要增加 -u updater.bin；且 adiff 工具的同级目录下需要有对应的 updater.bin，SDK 中提供了此 bin，也就是第 3 章中截图的 updater_NoPartition.bin，制作时将此 bin 拷贝到 adiff 同级目录后使用指令制作即可。

注意：使用 no updater 方案时，boot33 也需要使用 no updater 的 bin 文件，即 ql-config\quec-project\aboot\images 目录下的 boot33_NoLogo_NoUpdater.bin，将其更名为 boot33.bin 后即可。

3 带签名的 fota 功能介绍

1606S 8M 版本在 updater 对差分包做签名校验，支持 fbf 方式 fota；最小系统 mini fota 不支持。

如果是 4M 的项目，由于原本没有 updater 分区，所以默认不支持 fbf 方式的差分升级 kernel 方案，所以如果需要支持 fbf 方式的签名校验功能，需要客户自行增加 updater 分区和 fotapkg 分区（此分区大小 64K），随后将 ql-sdk\ql-config\quec-project\aboot\images 中对应的 updater_xxx.bin 更名为 updater.bin（因为打包时默认找的是 updater.bin）：
注：如果新增 updater 分区，由于 boot33 默认使用的是 no updater 方案，所以 boot33 也需要替换。

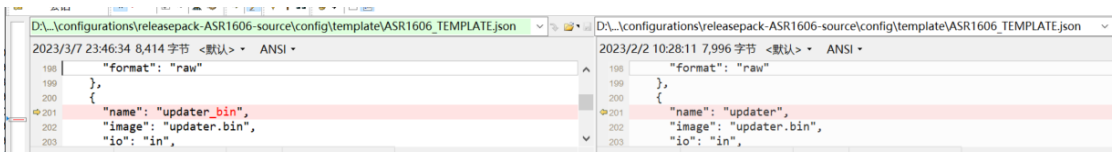
默认在 SDK 中 ql-config\quec-project\aboot\images 目录下我们提供了以下几个 updater.bin：

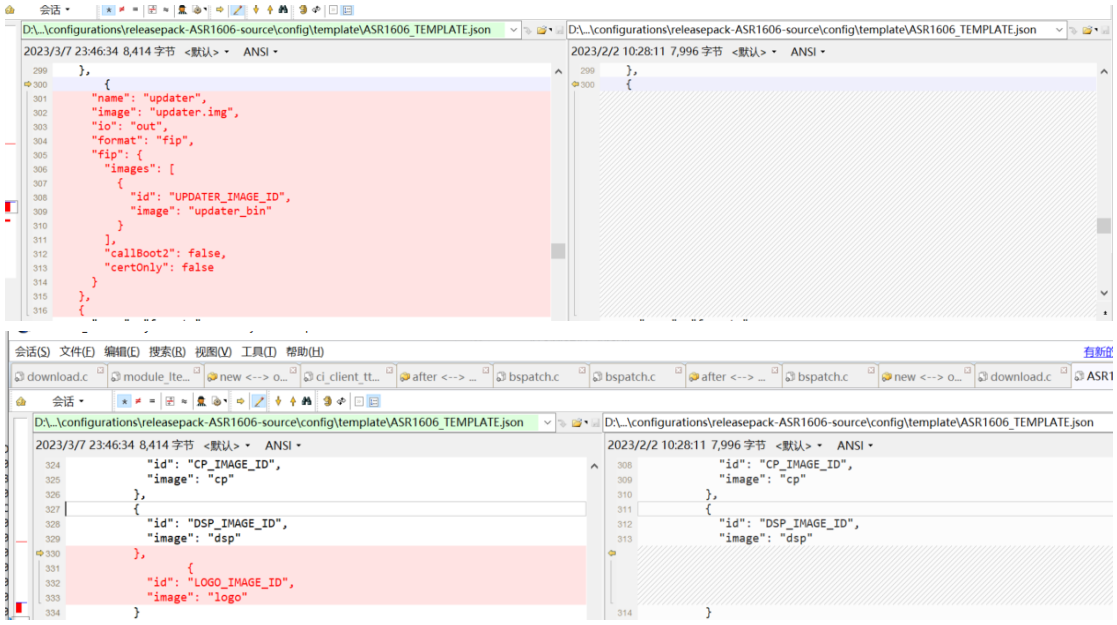
updater.bin	2023/3/23 10:15	BIN 文件	59 KB
updater_fbf.bin	2023/3/23 10:12	BIN 文件	84 KB
updater_NoPartition.bin	2023/2/27 13:31	BIN 文件	59 KB
updater_sec_fbf.bin	2023/3/23 10:12	BIN 文件	108 KB

Updater.bin：默认仅支持小系统升级、并且需要 updater 分区的 updater.bin；
Updater_fbf.bin：支持小系统、fbf 差分、全量升级，并且需要 updater 和 fota_pkg 分区的 updater.bin；
Updater_NoPartition.bin：仅支持小系统升级、并且不需要 updater 和 fota_pkg 分区的 updater.bin；
Updater_sec_fbf：支持小系统、fbf 差分、全量升级（仅 fbf 差分、全量支持签名校验功能），并且需要 updater 和 fota_pkg 分区的 updater.bin；

我们通过 FBF 工具制作出来的 fbf 升级包默认是不带签名功能的，如果需要增加签名功能，则需要进行以下步骤对升级包加签：

1. aboot config 使能 updater/logo 签名（在我们提供的 aboot 工具目录下）





2. 做出 fbf 差分包后，对差分包进行签名：（将差分包放到 aboot 根目录下并打开 cmd 命令行输入以下指令）

arelease -c ./ -s -p ASR_CRANEL_EVB -v CRANEL_A0_08MB --sign-from=fbf_dfota.bin fbf_dfota_cert.bin

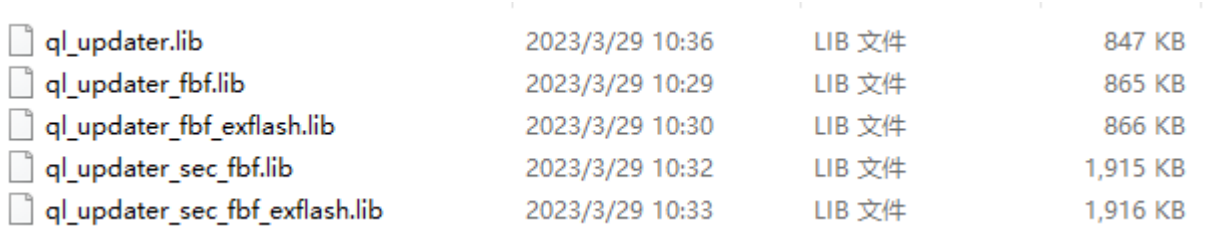
3. 经过上述后得到加签的升级包 fbf_dfota_cert.bin，后续升级时直接使用此 bin 即可；

4. 经过上述步骤后，升级包已经加过签名了，而我们默认的 updater.bin 是不带验签功能的，那么我们就需要替换成带验签功能的 updater.bin，也就是 ql-sdk\ql-config\quec-project\aboot\images 下的 updater_sec_fbf.bin，打包前需要将 updater_sec_fbf.bin 重命名为 updater.bin，然后打包出的固件在升级时就会包含验签功能，此时如果升级包中没有签名或包错误都会退出升级，回到正常系统运行。

4 客户定制 updater 说明

SDK 中除了默认带有第三章提供的几个 updater.bin 外，还在 ql-sdk\ql-bootloader\updater\lib 目录提供了几种不同功能的 ql_updater.lib，同直接提供的 bin 相比，这就能让客户更加自由的定制 updater 功能，比如 lcd 显示升级进度、使用外置 flash 挂载的文件系统等;

目前开放的 lib 有：



ql_updater.lib	2023/3/29 10:36	LIB 文件	847 KB
ql_updater_fbf.lib	2023/3/29 10:29	LIB 文件	865 KB
ql_updater_fbf_exflash.lib	2023/3/29 10:30	LIB 文件	866 KB
ql_updater_sec_fbf.lib	2023/3/29 10:32	LIB 文件	1,915 KB
ql_updater_sec_fbf_exflash.lib	2023/3/29 10:33	LIB 文件	1,916 KB

ql_updater.lib：默认使用的 lib，仅支持小系统升级功能，需要 updater 分区;

ql_updater_fbf.lib：支持小系统、fbf 差分、全量升级，并且需要 updater 和 fota_pkg 分区的 updater.bin（所有分区以及保存差分包的文件系统均需在内置 flash 上）;

ql_updater_fbf_exflash.lib：支持小系统、fbf 差分、全量升级，并且需要 updater 和 fota_pkg 分区的 updater.bin（保存差分包的文件系统支持放置在外挂 flash 上）;

ql_updater_sec_fbf.lib：支持小系统、fbf 差分、全量升级（仅 fbf 差分、全量支持签名校验功能），并且需要 updater 和 fota_pkg 分区的 updater.bin（所有分区以及保存差分包的文件系统均需在内置 flash 上）;

ql_updater_sec_fbf_exflash.lib：支持小系统、fbf 差分、全量升级（仅 fbf 差分、全量支持签名校验功能），并且需要 updater 和 fota_pkg 分区的 updater.bin（所有分区以及保存差分包的文件系统均需在内置 flash 上）;

当客户需要定制 updater 时，则可以找到上述对应功能的 lib，将其重命名为 ql_updater.lib（因为 makefile 中链接的是 ql_updater.lib，所以需要更名成这个名称，或者也可以更改 makefile 中链接的 lib 名称），随后找到 quectel/src 目录下的 ql_updater_usr_func.c，当前我们在此.c 中提供了上报升级进度的回调函数 ql_upgrade_percent_callback 以及设置外挂 flash 以及文件系统参数的接口 ql_set_external_fs_info，函数具体说明如下：

1、void ql_upgrade_percent_callback(unsigned int percent)

此接口在升级过程中会上报升级进度，进度由参数 percent 传递，此函数在升级过程中是阻塞的，即在该函数中的耗时会累加到升级过程中。

客户可按照当前.c 中的写法作为参考实现，比如客户需要使用 lcd 显示进度，那么根据 percent 进行打印，不过需要注意的是我们字啊 updater 中默认不提供 lcd 驱动，客户可从 kernel 侧挪过来直接使用即；升级错误也会上报错误码。

2、void ql_set_external_fs_info(int *port, int *clk, unsigned int *start_addr, unsigned int *size)

此接口仅在客户使用外挂 flash 并挂载非 customer_fs 系统时使用（比如客户 customer_fs 挂载在内置 flash，但由于空间不足，客户又外挂了 flash 并使用我们 cp 侧的 qextfs_init 接口挂载了文件系统，此时就需要使用此接口进行配置参数）

Port: 0: gpio16~19 1/2: gpio33~36 3: gpio 12~15

Clk: spi 时钟速率：

#define _APBC_SSP_FNCLKSEL_6_5MHZ (0x00)
#define _APBC_SSP_FNCLKSEL_13MHZ (0x01)
#define _APBC_SSP_FNCLKSEL_26MHZ (0x02)
#define _APBC_SSP_FNCLKSEL_52MHZ (0x03)
#define _APBC_SSP_FNCLKSEL_3_25MHZ (0x04)
#define _APBC_SSP_FNCLKSEL_1_625MHZ (0x05)
#define _APBC_SSP_FNCLKSEL_812_5KHZ (0x06)

start_addr：文件系统挂载的起始地址，同 cp 侧 qextfs_init 填入的 startaddr 必须相同，否则会挂载失败，无法升级。

Size: 文件系统的大小，必须同 cp 侧 qextfs_init 填入的 size 相同，否则挂载失败，无法进行升级。

当客户未更改此接口时，默认使用的是 port1:gpio33~36，26MHZ 的 spi，从外置 flash 地址 0 还是大小为 0x100000 的文件系统。客户更改的话也只需要按照已有的接口改变对应的值即可。

5 Fota 功能测试

SDK 中我们提供了 example_fota.c，该.c 中我们提供了三组测试函数：小系统测试、单独 app 升级测试、fbf 方式带签名/不带签名的差分、全量升级方式。

小系统测试和单独 app 升级测试比较简单，只要填入的 url 正确，就可一健进行升级，下面主要介绍下 fbf 方式升级测试 quec_fota_uart_test：

```
static void quec_fota_uart_test(void * argv)
{
    int ret = -1,event = 0;
    int write_bytes = 0;
    unsigned char data[] = "hello, this is file fota test!!!, please Transfer dfota.bin\n";
    unsigned char read_buf[4096] = {0};
    ql_uart_config_t dcb;
    int count = 0, res = 0,file_len = 0;
    QFILE *fp = NULL;
    qlFotaImgProcCtxPtr ctx = NULL;
    int file_size = 0;
#ifdef QUEC_EXTERNAL_NORFLASH_TEST
    unsigned char *id;
    int port_index = EXTERNAL_NORFLASH_PORT33_36;
    int clk = _APBC_SSP_FNCLKSEL_1_625MHZ_;

    printf("[FS] ========== exflash init  \r\n");
    ql_spi_nor_init(port_index, clk);

    id = ql_spi_nor_read_id(port_index);
    printf("[NORFLASH]=====norflash ID:0x%02x 0x%02x 0x%02x\r\n", id[0], id[1], id[2]);

    ret = qextfs_init('C', "external_fs", 1, port_index, 0, 0x100000);
    printf("[FS] ========== exfs init : %d  \r\n", ret);

    ql_fota_set_package_path("C:/FotaFile.bin");
    ql_remove("C:/FotaFile.bin");
#endif
    ret = ql_uart_open(QL_MAIN_UART_PORT, QL_UART_BAUD_115200, QL_FC_HW);
    if (ret) { ...

    ql_rtos_flag_create(&fota_uart_flag);
    /*****************************************/
    ql_uart_get_dcbconfig(QL_MAIN_UART_PORT, &dcb);
    printf("[%s][%d]baudrate:%d, data_bit:%d, stop_bit:%d, parity_bit:%d, flow_ctrl:%d \r\n", __func__, __LINE__, dcb.bi
    dcb.baudrate = QL_UART_BAUD_115200;
    ql_uart_set_dcbconfig(QL_MAIN_UART_PORT, &dcb);
    /*****************************************/

    ql_uart_register_cb(QL_MAIN_UART_PORT, quec_fota_uart_callback);    //use callback to read uart data


    ql_uart_write(QL_MAIN_UART_PORT, data, sizeof(data));
    ql_rtos_flag_wait(fota_uart_flag, QL_FOTA_UART_READ, QL_FLAG_OR, &event, QL_WAIT_FOREVER);

    ctx = ql_fota_init();
    if(!ctx) ...

    count = 0;
    while((ret >= 0) && (count < 15)){ ...

    if(count >= 15){
        printf("Transfer end:%d ,file_len:%d\n",ret, file_len);
    }
    ql_uart_close(QL_MAIN_UART_PORT);

    ret = ql_fota_image_flush(ctx);
    if(ret)
    {
        printf("*** fota image flush fail ***");
        return;
    }
    printf("ql_fota_image_flush end\n");
    ret = ql_fota_image_verify(ctx);
    if(ret) ...
    printf("ql_fota_image_verify success! \n");


    printf("ready to power reset!!\n");
    ql_power_reset();
} « end quec_fota_uart_test »
```

该函数使用的 fota 接口为 ql_fota_init/ql_fota_image_write 等这一组，该组接口获取差分包由客户自行进行，无论是从服务器下载、通过串口传输等方式，只要通过 ql_fota_image_write 写到文件系统后校验通过，下次重启就能进入升级，而我们的测试用例就是通过串口传输方式实现差分包的获取。

如果客户是外挂的文件系统（仅针对有 customer_fs 下另外挂载的文件系统），则需要打开宏 QUEC_EXTERNAL_NORFLASH_TEST，修改 spi、文件系统配置信息（此处的信息必须同第 4 章的 ql_set_external_fs_info 函数参数保持一致），**注意一定要调用 ql_fota_set_package_path 接口设置差分包存储位置**（如果不使用此接口设置，则差分包默认下载到 customer_fs.bin 中的 U:/FotaFile.bin），比如外挂的是 C 盘，则将此名称修改为"C:/FotaFile.bin"，这样后续下载、检验都是在外置 flash 的文件系统中进行。